

EXERCISE 04- SQL FOUNDATIONS - JOINS

TABLE CREATION:

▶ Run all (1000) ▼ ✓ 07:54 PM (<1s) workspace default ▼ Last edit was 7 minutes ago

Edit

Serverless Starte... 2XS ▼

Schedule

Save*

```
1 | create CATALOG IF NOT EXISTS joins;
2 | create SCHEMA IF NOT EXISTS joins.bl;
3 | --CREATE TABLE 1--
4 | create TABLE joins.bl.users (user_id INT, user_name STRING, country STRING);
5 | insert into joins.bl.users values (1, 'Nomvula', 'Johannesburg'), (2, 'David', 'Cape Town'), (3, 'Anele', 'Durban'), (4, 'Kabelo', 'Pretoria'), (5, 'Lerato', 'Port Elizabeth');
6 |
7 | SELECT * FROM joins.bl.users;
8 | --CREATE TABLE 2
9 | CREATE TABLE joins.bl.plans (plan_id INT, plan_name STRING, monthly_price INT);
10 | insert into joins.bl.plans values (10, 'Basic', 79), (11, 'Standard', 129), (12, 'Premium', 199), (13, 'Family', 249), (14, 'Mobile', 59);
11 |
12 | SELECT * FROM joins.bl.plans;
13 |
14 | --CREATE TABLE 3
15 | CREATE TABLE joins.bl.subscriptions (subscription_id INT, user_id INT, plan_id INT, start_date DATE);
16 | insert into joins.bl.subscriptions values (501, 1, 10, '2026-01-15'), (502, 2, 11, '2026-02-01'), (503, 1, 12, '2026-03-10'), (504, 6, 11, '2026-03-20'), (505, 3, 13, '2026-04-05');
17 | SELECT * FROM joins.bl.subscriptions;
18 |
19 | --CREATE TABLE 4
20 | CREATE TABLE joins.bl.shows (show_id INT, show_title STRING, genre STRING);
21 | insert into joins.bl.shows values (701, 'Comedy hour', 'Comedy'), (702, 'Crime Time', 'Drama'), (703, 'Tech Tales', 'Documentary'), (704, 'Cooking Lab', 'Lifestyle'), (705, 'Wild E
    'Documentary');
22 | SELECT * FROM joins.bl.shows;
23 |
24 | --CREATE TABLE 5
25 | CREATE TABLE joins.bl.viewing_sessions (session_id INT, user_id INT, show_id INT, watch_minutes INT);
26 | insert into joins.bl.viewing_sessions values (901, 1, 701, 45), (902, 2, 703, 30), (903, 1, 702, 60), (904, 7, 701, 20), (905, 3, 705, 90);
27 | SELECT * FROM joins.bl.viewing_sessions;
28 | --INNER JOIN--
```

Question 01:Show every user who has a subscription.
Match users to subscriptions(user_id, user_name,
subscription_id, start_date)

```

29
30 --Question 01: Show every user who has a subscription. Match users to subscriptions.
31
32 SELECT A.user_id, A.user_name, B.subscription_id, B.start_date
33 FROM joins.bl.users AS A
34 INNER JOIN joins.bl.subscriptions AS B
35 ON A.user_id = B.user_id;
36
37

```

Add parameter

Table ▾ +

	1 ² ₃ user_id	A ^B _C user_name	1 ² ₃ subscription_id	📅 start_date
1	1	Nomvula	503	2026-03-10
2	2	David	502	2026-02-01
3	3	Anele	505	2026-04-05
4	1	Nomvula	501	2026-01-15

Question 02:Show every subscription with its matching
plan name and monthly price (subscription_id, user_id,
plan_name, monthly_price)

```

36
37 --Question 02:Show every subscription with its matching plan name and monthly price.
38
39 SELECT A.subscription_id, A.user_id, B.plan_name, B.monthly_price
40 FROM joins.bl.subscriptions AS A
41 INNER JOIN joins.bl.plans AS B
42 ON A.plan_id = B.plan_id;
43

```

Add parameter

Table ▾ +

	1 ² ₃ subscription_id	1 ² ₃ user_id	A ^B _C plan_name	1 ² ₃ monthly_price
1	501	1	Basic	79
2	504	6	Standard	129
3	503	1	Premium	199
4	505	3	Family	249
5	502	2	Standard	129

Question 03: Show every viewing session that has a matching show. Include the show title and genre (session_id, user_id, show_title, genre, watch_minutes)

```

43
44 --Question 03: Show every viewing session that has a matching show. Include the show title and genre.
45 SELECT A.session_id, A.user_id, B.show_title, B.genre, A.watch_minutes
46 FROM joins.b1.viewing_sessions AS A
47 INNER JOIN joins.b1.shows AS B
48 ON A.show_id = B.show_id;

```

Add parameter

Table +

	session_id	user_id	show_title	genre	watch_minutes
1	904	7	Comedy hour	Comedy	20
2	903	1	Crime Time	Drama	60
3	902	2	Tech Tales	Documentary	30
4	901	1	Comedy hour	Comedy	45

Question 04: Show every viewing session with the user who watched it. Only show sessions with a matching user(user_name, country, session_id, show_id, watch_minutes)

```

49
50 --Question 04: Show every viewing session with the user who watched it. Only show sessions with a matching user(user_name, country, session_id, show_id,
    watch_minutes)
51
52 SELECT A.user_name, A.country, B.session_id, B.show_id, B.watch_minutes
53 FROM joins.b1.users AS A
54 INNER JOIN joins.b1.viewing_sessions AS B
55 ON A.user_id = B.user_id;
56
57

```

Add parameter

Table +

	user_name	country	session_id	show_id	watch_minutes
1	Nomvula	Johannesburg	903	702	60
2	David	Cape Town	902	703	30
3	Anele	Durban	905	705	90
4	Nomvula	Johannesburg	901	701	45

Question 05: Show users along with their subscriptions, the plan name, and the price. Use only users who have both a subscription and a valid plan(user_name, country, plan_name, monthly_price, start_date)

```
57 --Question 05: Show users along with their subscriptions, the plan name, and the price. Use only users who have both a subscription and a valid plan(user_name,
country, plan_name, monthly_price, start_date)
58 SELECT A.user_name, A.country, C.plan_name, C.monthly_price, B.start_date
59 FROM joins.bl.users AS A
60 INNER JOIN joins.bl.subscriptions AS B
61 ON A.user_id = B.user_id
62 INNER JOIN joins.bl.plans AS C
63 ON B.plan_id = C.plan_id;
64
65
66
```

Add parameter

Table +

Q Y I □ ⌂ ^ X

	^A _C user_name	^A _C country	^A _C plan_name	¹ ₃ monthly_price	¹ ₃ start_date
1	Nomvula	Johannesburg	Premium	199	2026-03-10
2	David	Cape Town	Standard	129	2026-02-01
3	Anele	Durban	Family	249	2026-04-05
4	Nomvula	Johannesburg	Basic	79	2026-01-15

PART B: LEFT JOIN

Question 06: Show every user and any subscriptions they have. Users without subscriptions must still appear (user_id, user_name, subscription_id, start_date)

```
65 --PART B :LEFT JOIN
66
67 --Question 6:Show every user and any subscriptions they have. Users without subscriptions must still appear (user_id, user_name, subscription_id, start_date)
68
69 SELECT A.user_id, A.user_name, B.subscription_id, B.start_date
70 FROM joins.bl.users AS A
71 LEFT JOIN joins.bl.subscriptions AS B
72 ON A.user_id = B.user_id;
```

Add parameter

Table +

Q Y I □ ⌕ ^ X

	user_id	user_name	subscription_id	start_date
1	1	Nomvula	503	2026-03-10
2	2	David	502	2026-02-01
3	3	Anele	505	2026-04-05
4	4	Kabelo	null	null
5	5	Lerato	null	null
6	1	Nomvula	501	2026-01-15

Question 07: Show every plan and the subscriptions on it. Plans with no subscribers must still appear(plan_id, plan_name, subscription_id, user_id)

```

75
76 --Question 7: Show every plan and the subscriptions on it. Plans with no subscribers must still appear(plan_id, plan_name, subscription_id, user_id)
77
78 SELECT A.plan_id, A.plan_name, B.subscription_id, B.user_id
79 FROM joins.bl.plans AS A
80 LEFT JOIN joins.bl.subscriptions AS B
81 ON A.plan_id = B.plan_id;
82
83
84
85

```

Id parameter

plan_id	plan_name	subscription_id	user_id
10	Basic	501	1
11	Standard	504	6
12	Premium	503	1
13	Family	505	3
14	Mobile	null	null
11	Standard	502	2

Question 08: Show every show and any viewing sessions on it. Shows that were never watched must still appear (show_id, show_title, session_id, watch_minutes)

```

83
84 --Question 8: Show every show and any viewing sessions on it. Shows that were never watched must still appear(show_id, show_title, session_id, watch_minutes)
85
86 SELECT A.show_id, A.show_title, B.session_id, B.watch_minutes
87 FROM joins.bl.shows AS A
88 LEFT JOIN joins.bl.viewing_sessions AS B
89 ON A.show_id = B.show_id;
90
91

```

Add parameter

show_id	show_title	session_id	watch_minutes
701	Comedy hour	904	20
702	Crime Time	903	60
703	Tech Tales	902	30
704	Cooking Lab	null	null
706	Wild Earth	null	null
701	Comedy hour	901	45

Question 09: Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear (with NULL user details) (session_id, show_id, watch_minutes, user_id, user_name)

```
91 --Question 09: Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear (with NULL user details)
92 (session_id, show_id, watch_minutes, user_id, user_name)
93 SELECT A.session_id, A.show_id, A.watch_minutes, B.user_id, B.user_name
94 FROM joins.bl.viewing_sessions AS A
95 LEFT JOIN joins.bl.users AS B
96 ON A.user_id = B.user_id;
97
98
```

Add parameter

Table +

Q Y H O S ^ X

	session_id	show_id	watch_minutes	user_id	user_name
1	901	701	45	1	Nomvula
2	902	703	30	2	David
3	903	702	60	1	Nomvula
4	904	701	20	null	null
5	905	705	90	3	Anele

Question 10: Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear (user_name, country, plan_name, monthly_price)

```

98  --Question 10: Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear (user_name, country,
99  plan_name, monthly_price)
100 SELECT A.user_name, A.country, C.plan_name, C.monthly_price
101 FROM joins.bl.users AS A
102 LEFT JOIN joins.bl.subscriptions AS B
103 ON A.user_id = B.user_id
104 LEFT JOIN joins.bl.plans AS C
105 ON B.plan_id = C.plan_id;

```

Add parameter

Table ▾ +

Q Y H □ E ^

	A ₀ user_name	A ₀ country	A ₀ plan_name	I ₃ monthly_price
1	Nomvula	Johannesburg	Premium	199
2	David	Cape Town	Standard	129
3	Anele	Durban	Family	249
4	Kabelo	Pretoria	null	null
5	Lerato	Port Elizabeth	null	null
6	Nomvula	Johannesburg	Basic	79

PART C: FULL OUTER JOIN

Question 11: Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist (user_id, user_name, subscription_id, start_date)

```
107 --Question 11::Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist(user_id,
108 user_name, subscription_id, start_date)
109 SELECT A.user_id,A.user_name,B.subscription_id,B.start_date
110 FROM joins.bl.users AS A
111 FULL OUTER JOIN joins.bl.subscriptions AS B
112 ON A.user_id=B.user_id;
113
```

Add parameter

	user_id	user_name	subscription_id	start_date
1	5	Lerato	null	null
2	1	Nomvula	503	2026-03-10
3	3	Anele	505	2026-04-05
4	2	David	502	2026-02-01
5	4	Kabelo	null	null
6	1	Nomvula	501	2026-01-15
7	null	null	504	2026-03-20

Question 12: Show every plan and every subscription, including plans without subscribers AND any subscription referencing a plan that does not exist (plan_id, plan_name, subscription_id, user_id)

```
114 --Question 12: Show every plan and every subscription, including plans without subscribers AND any subscription referencing a plan that does not exist(plan_id,  
115 plan_name, subscription_id, user_id)  
116 SELECT A.plan_id, A.plan_name, B.subscription_id, B.user_id  
117 FROM joins.bl.plans AS A  
118 FULL OUTER JOIN joins.bl.subscriptions AS B  
119 ON A.plan_id=B.plan_id;  
120
```

Add parameter

Table ▾ +

Q Y | □ ⚙ ^ X

	plan_id	plan_name	subscription_id	user_id
1	12	Premium	503	1
2	10	Basic	501	1
3	13	Family	505	3
4	14	Mobile	null	null
5	11	Standard	504	6
6	11	Standard	502	2

Question 13:Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist (show_id, show_title, session_id, watch_minutes)

```
120
121 --Question 13:Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist(show_id,
    show_title, session_id, watch_minutes)
122
123 SELECT A.show_id,A.show_title,B.session_id,B.watch_minutes
124 FROM joins.bl.shows AS A
125 FULL OUTER JOIN joins.bl.viewing_sessions AS B
126 ON A.show_id=B.show_id;
127
```

Add parameter

Table +

	show_id	show_title	session_id	watch_minutes
1	704	Cooking Lab	null	null
2	703	Tech Tales	902	30
3	701	Comedy hour	904	20
4	702	Crime Time	903	60
5	706	Wild Earth	null	null
6	701	Comedy hour	901	45
7	null	null	905	90

Question 14: Show every user and every viewing session, including users with no sessions AND sessions referencing users who do not exist (user_id, user_name, session_id, show_id, watch_minutes)

```
127
128 --Question 14: Show every user and every viewing session, including users with no sessions AND sessions referencing users who do not exist (user_id, user_name,
    session_id, show_id, watch_minutes)
129
130 SELECT A.user_id, A.user_name, B.session_id, B.show_id, B.watch_minutes
131 FROM joins.bl.users AS A
132 FULL OUTER JOIN joins.bl.viewing_sessions AS B
133 ON A.user_id=B.user_id;
```

Add parameter

Table ▾ +

Q Y I □ E ^ X

	1 ² ₃ user_id	A ₀ user_name	1 ² ₃ session_id	1 ² ₃ show_id	1 ² ₃ watch_minutes
1	1	Nomvula	901	701	45
2	1	Nomvula	903	702	60
3	3	Anele	905	705	90
4	2	David	902	703	30
5	null	null	904	701	20
6	4	Kabelo	null	null	null
7	5	Lerato	null	null	null

Question 15: Show every user, every subscription, and every plan in one query — using FULL OUTER JOIN throughout. This is the hardest question — get all gaps visible at once(user_id, user_name, subscription_id, plan_id, plan_name)

135 --Question 15:Show every user, every subscription, and every plan in one query - using FULL OUTER JOIN throughout. This is the hardest question - get all gaps
visible at once(user_id, user_name, subscription_id, plan_id, plan_name)

136

137 SELECT A.user_id,A.user_name,B.subscription_id,B.plan_id,C.plan_name

138 FROM joins.bl.users AS A

139 FULL OUTER JOIN joins.bl.subscriptions AS B

140 ON A.user_id=B.user_id

141 FULL OUTER JOIN joins.bl.plans AS C

142 ON B.plan_id=C.plan_id;

Add parameter

Table +

	user_id	user_name	subscription_id	plan_id	plan_name
1	1	Nomvula	503	12	Premium
2	1	Nomvula	501	10	Basic
3	3	Anele	505	13	Family
4	5	Lerato	null	null	null
5	4	Kabelo	null	null	null
6	2	David	502	11	Standard
7	null	null	504	11	Standard
8	null	null	null	null	Mobile